Experiment 3

SystemVerilog Interfaces (Clocked 2×1 Multiplexer Verification Platform)

AIM:

To understand and familiarize with the usage of **SystemVerilog interfaces**, **modports**, and **clocking blocks** for building a verification platform.

SOFTWARE/TOOLS USED: Synopsys VCS

DUT: The devices under test:

1. Clocked 2×1 Multiplexer Block
    a. It works with an active low asynchronous reset
    b. The output is registered at the rising edge of the clock.

Questions:

- Design an interface named **mux_if** that contains all signals including clock and reset.
    - The interface should contain modports for DUT and TEST
    - The interface should have a clocking block with sampling and driving skews
- Design a clocked 2×1 multiplexer module using the interface designed above.
- Design a testbench that
    - Instantiates the interface
    - Instantiates the DUT and connects it using the interface
    - Generates clock through the interface
    - Applies reset and test stimulus through the interface
    - Has an inbuilt checker that tests if the output values from the multiplexer matches with the expected value and produces a "PASS" or "FAIL" message.

The skeleton of the testbench given below may be used.

```
module mux_tb;
 // Instantiate the interface
 // mux_if mif();

 // Instantiate the DUT and connect to interface
 // mux2_1_clk dut( ... );

 // Clock generation
 // initial begin
 //     mif.clk = 0;
 //     forever #5 mif.clk = ~mif.clk;
 // end

 initial begin
    // Reset the system
    // Hold reset for 10 clock cycles
    // Release reset

    // Apply test cases
    // Test case 1
    // Drive a, b, sel
    // Wait for posedge clk
    // Calculate expected
    // Display y and PASS/FAIL status

    // Test case 2
    // Test case 3
    // ...

    $finish;
 end
```

THEORY:

When digital designs become complex, connecting many signals directly between the testbench and the design makes verification hard to manage. SystemVerilog provides interfaces, modports, and clocking blocks to make these connections easier and more organized. They help control how signals are accessed, maintain proper timing, and make the testbench easier to understand, reuse, and maintain.

1. **Clocked 2×1 Multiplexer**

A 2×1 multiplexer is a digital circuit that selects one of two input signals based on the value of a select line. When the select line is low, the output follows the first input, and when the select line is high, the output follows the second input. In this experiment, the multiplexer is implemented as a clocked and registered circuit, so the output updates only on the positive edge of the clock. An active-low asynchronous reset is also used, which forces the output to zero whenever the reset signal is low, regardless of the clock.

2. **SystemVerilog interface**

A SystemVerilog interface is used to group related signals into a single unit and connect them between the design under test and the testbench. This reduces the number of individual ports, makes the code easier to read, and allows the same interface to be reused in different testbenches. Interfaces also help maintain proper signal direction and access control through the use of modports. In this experiment, the interface named mux_if includes the data inputs in[1:0], the select signal sel, the output out, the clock signal clk, and the reset signal reset.

3. **Modports**

Modports are used within a SystemVerilog interface to define how different components access the signals. In this experiment, one modport is used by the multiplexer design, where the data inputs, select signal, reset, and clock are treated as inputs and the output signal is driven by the design. Another modport is used by the testbench, which drives the input and reset signals and observes the output. Using modports prevents signals from being driven in the wrong direction and helps maintain clear separation between the design and the testbench.

4. **Clocking blocks**

A clocking block is used to synchronize when signals are driven and when they are sampled with respect to the clock edge. It helps avoid race conditions and ensures that timing behavior is clear and predictable. In this experiment, the input signals such as in, sel, and reset are driven slightly after the clock edge, while the output signal out is sampled slightly after the same clock edge. This closely represents real hardware behavior and results in reliable verification.

5. **Testbench**

A self-checking testbench is designed to automatically verify the correctness of the design. The testbench first applies a reset through the interface, then drives different combinations of the input and select signals. It waits for the appropriate clock edges, calculates the expected output internally, and compares it with the output produced by the design under test. Based on this comparison, the testbench displays a PASS or FAIL message, creating a fully automated and efficient verification environment.

SYSTEM VERILOG CODE WITH COMMENTS (#):

INTERFACE:

```
interface mux_if(input bit clk);
	bit rst;
	bit a, b, sel;
	bit y;

	clocking cb@(posedge clk);
		default input #1step output #2;
		output a, b, sel;
		input y;
	endclocking

	modport TEST(
		clocking cb,
		output rst,
		input clk
		);

	modport DUT(
		input a, b, sel, rst, clk,
		output y
		);
endinterface
```

DUT:

```
module mux2_1_clk(mux_if.DUT mif);
always @(posedge mif.clk or negedge mif.rst)
begin
	if(!mif.rst)
		mif.y <= 1'b0;
	else begin
		if (mif.sel == 1'b0) mif.y <=
mif.a;
		else mif.y <= mif.b;
	end
end

endmodule
```

TESTBENCH:

```
module mux_tb(mux_if.TEST mif);

int i;

initial begin
// Reset the system
// Hold reset for 10 clock cycles
// Release reset
	mif.rst <= 1'b0;
	repeat(10) @mif.cb;
	mif.rst <= 1'b1;

	mif.cb.sel<=1'b0;
	mif.cb.a<=1'b0;
	mif.cb.b<=1'b0;

// Apply test cases
	for (i = 0; i < 8; i++) begin
		@mif.cb;
		mif.cb.a   <= i[0];
		mif.cb.b   <= i[1];
		mif.cb.sel <= i[2];

		$display("@%0t: a = %0b, b =
%0b, sel = %0b", $time, i[0], i[1], i[2]);

// Wait for DUT to update
		repeat(2) @mif.cb;

// Calculate expected
// Display y and PASS/FAIL status
		if (i[2] == 1'b0) begin
			if (mif.cb.y != i[0])

	$display("@%0t: ERROR: When sel =
0, y != a", $time);
			else

	$display("@%0t: PASS: When sel = 0,
y = a", $time);
		end
		else begin
			if (mif.cb.y != i[1])

	$display("@%0t: ERROR: When sel =
1, y != b", $time);
			else

	$display("@%0t: PASS: When sel = 1,
y = b", $time);
		end
	end
	$finish;
end
endmodule
```

TOP MODULE:

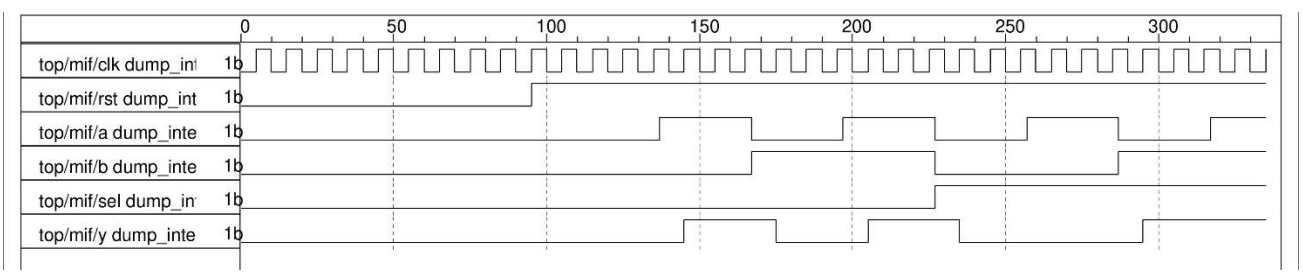
```
module top;
        bit clk=0;
        always #5 clk = ~clk;
        mux_if mif(clk);
        mux2_1_clk dut(mif);
        mux_tb t1(mif);
        initial begin
                $dumpfile("dump_interf.vcd");
                $dumpvars(0,top);
        end
endmodule
```

RESULTS SIMULATIONS/OUTPUT STATEMENTS:

Output from Transcript window:

```
Compiler version V-2023.12-1_Full64; Runtime version V-2023.12-1_Full64;  Jan 24 05:24 2026
@105: a = 0, b = 0, sel = 0
@125: PASS: when sel = 0, y = a
@135: a = 1, b = 0, sel = 0
@155: PASS: when sel = 0, y = a
@165: a = 0, b = 1, sel = 0
@185: PASS: when sel = 0, y = a
@195: a = 1, b = 1, sel = 0
@215: PASS: when sel = 0, y = a
@225: a = 0, b = 0, sel = 1
@245: PASS: when sel = 1, y = b
@255: a = 1, b = 0, sel = 1
@275: PASS: when sel = 1, y = b
@285: a = 0, b = 1, sel = 1
@305: PASS: when sel = 1, y = b
@315: a = 1, b = 1, sel = 1
@335: PASS: when sel = 1, y = b
$finish called from file "mux_tb.sv", line 94.
$finish at simulation time                   335
```

Waveform output:



Inference:

The experiment successfully demonstrated the design and verification of a clocked 2×1 multiplexer using SystemVerilog interfaces, modports, and clocking blocks. Simulation results along with the self-checking testbench verified that the output updates correctly on every positive clock edge according to the select signal. The active-low reset was confirmed to bring the output to a known initial state. The use of interface-based verification improved signal management, ensured proper timing coordination, and made the testbench easier to understand. Hence, the intended functionality of the clocked 2×1 multiplexer has been validated.